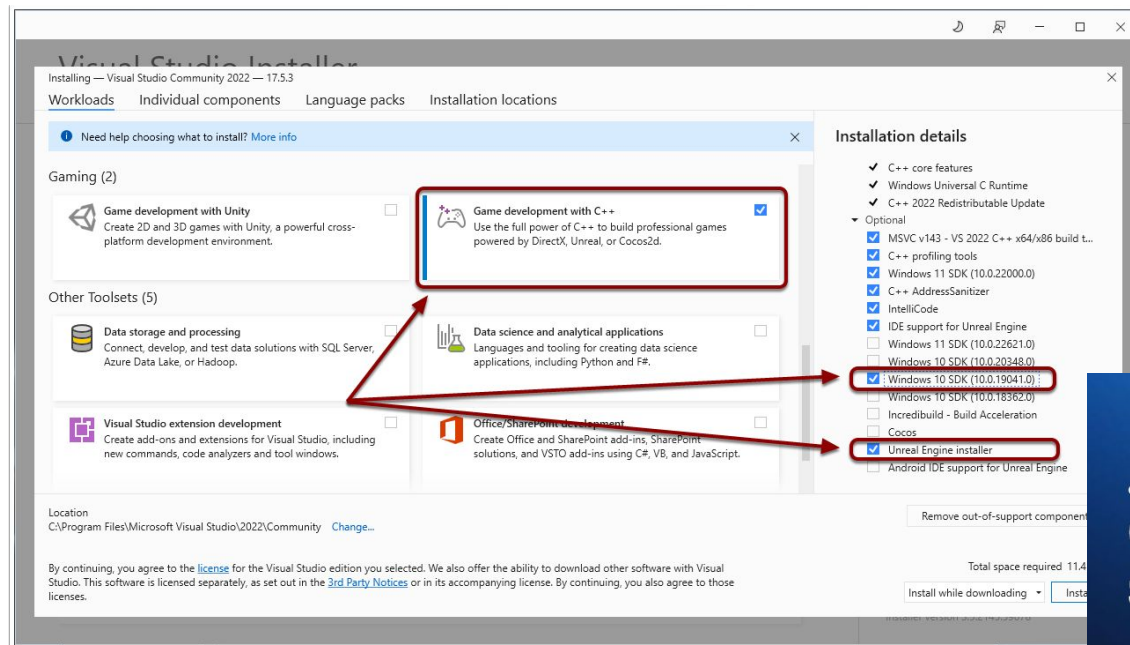




Unreal Engine

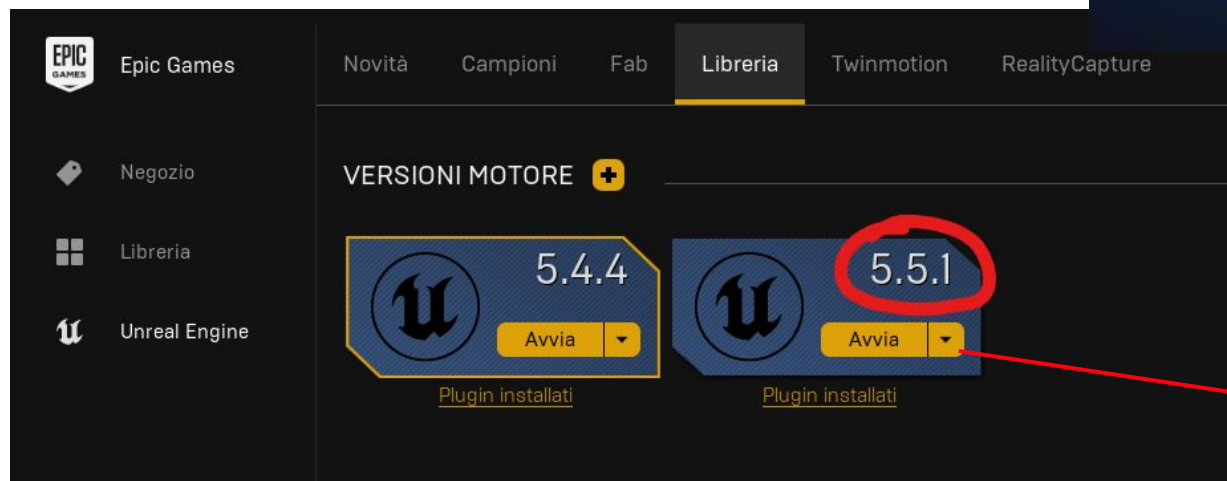


ORA SU DISPOSITIVI MOBILI

Ottieni l'Epic Games Store

[Installa su Windows](#)

Disponibile anche su macOS, Android, iPhone (solo UE) e iPad (solo UE).



Opzioni di installazione di Unreal Engine 5.5.1 ✕

Componenti di base (Richiesto)	34,28 GB	✓
Contenuto iniziale	641,71 MB	✓
Template e pacchetti di funzioni	3,77 GB	✓
Engine Source	385,04 MB	✓
Simboli dell'editor per il debug	38,36 GB	✓
► Piattaforme di destinazione		✗

Dimensioni download: 0,00 B

Spazio di memorizzazione necessario: 77,44 GB



Unreal Engine

Unreal Engine (**UE**) è un motore grafico di sviluppo di videogiochi e applicazioni interattive creato da **Epic Games**. È uno degli strumenti più avanzati e versatili disponibili nel settore del game development, utilizzato per creare esperienze interattive in 2D, 3D e realtà virtuale (VR).

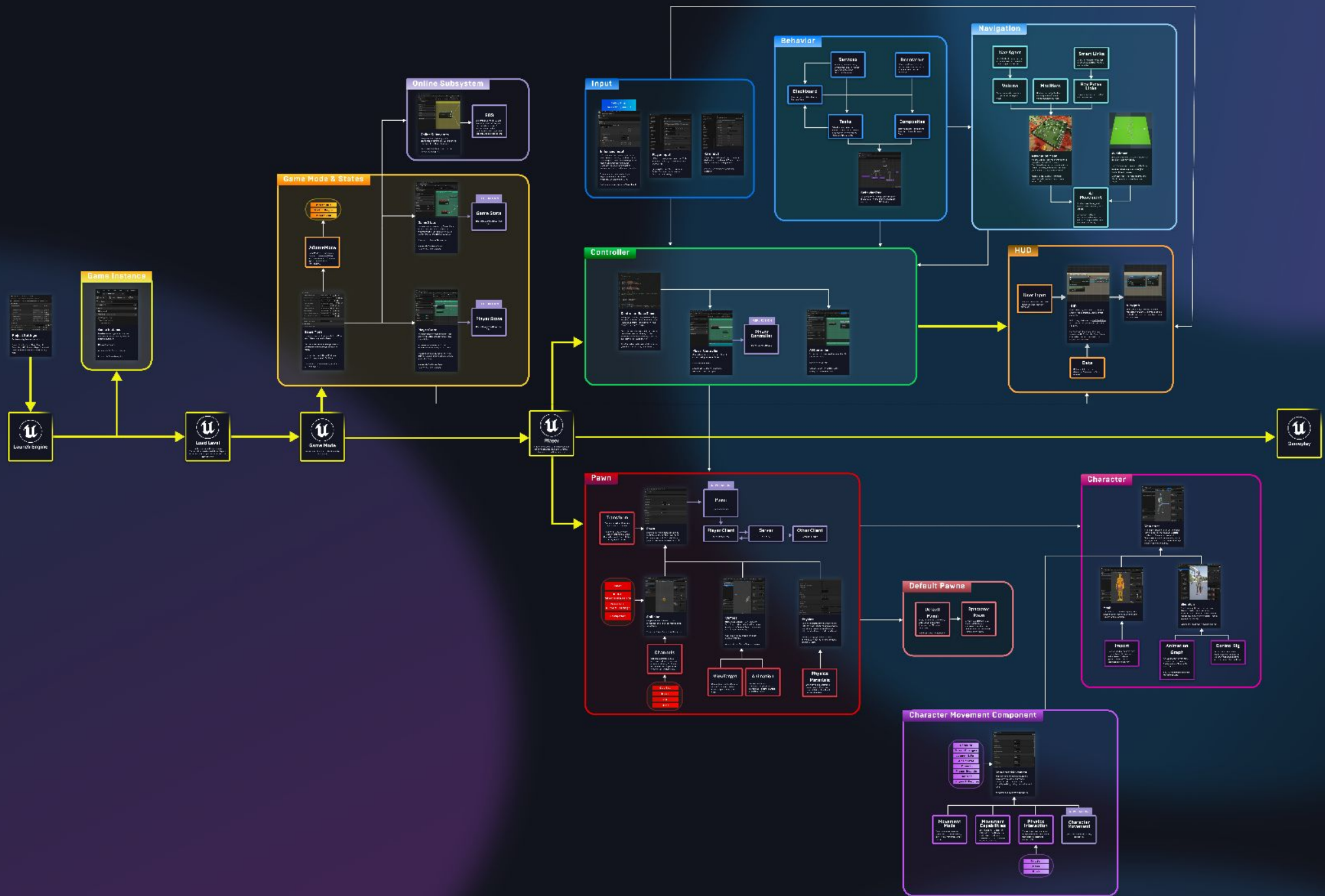
UE offre un sistema preconfigurato chiamato Gameplay Framework (**GF**), che include molte funzionalità necessarie per lo sviluppo di un videogioco; questo sistema copre tutto, da un avanzato sistema di input a punti di accesso comuni che permettono di accedere facilmente ai dati o allo stato del gioco.



Unreal Engine Gameplay Framework

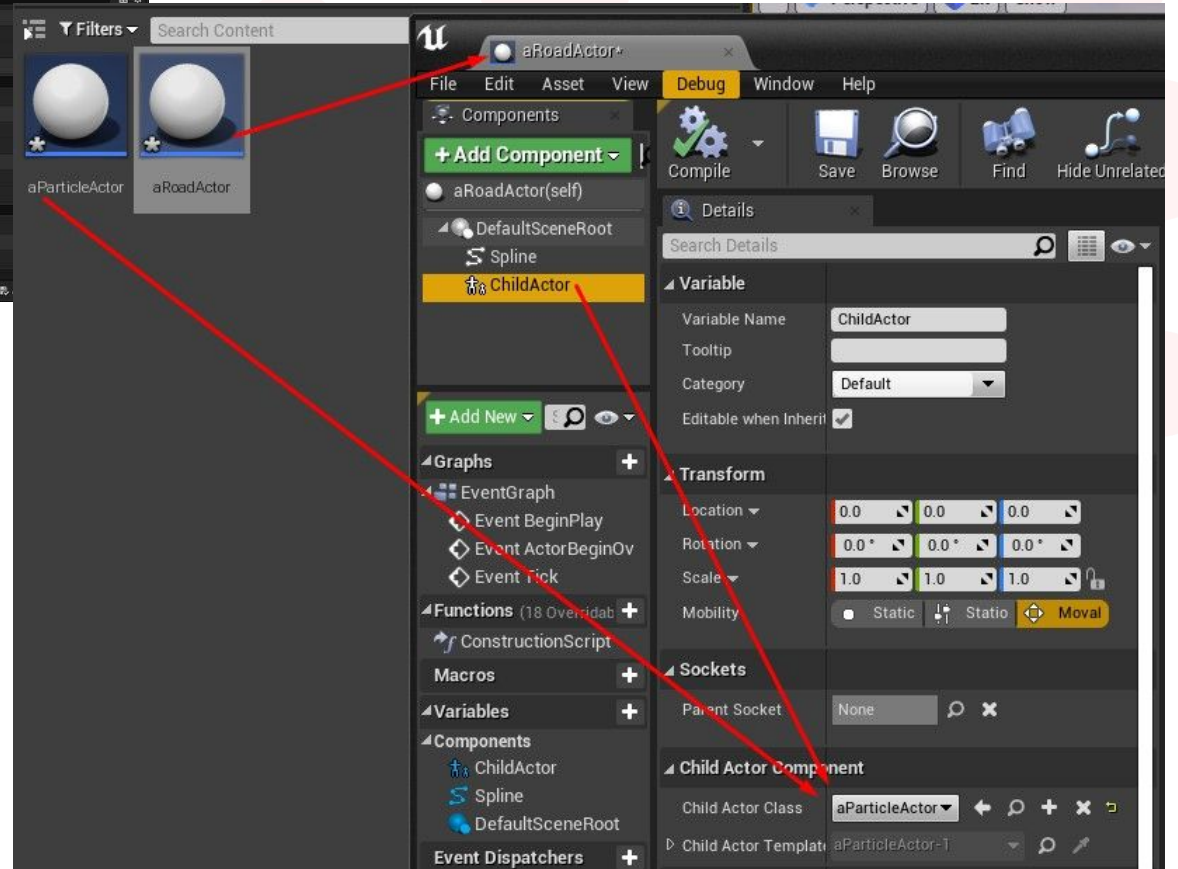
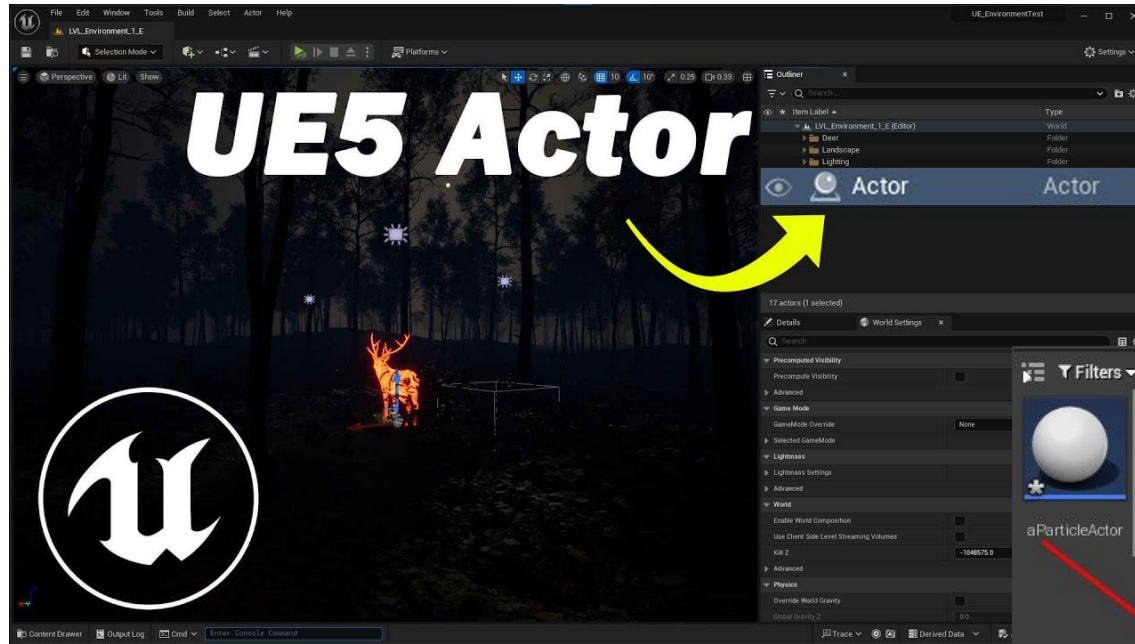
Ecco alcuni punti chiave che spiegano perché il GF è così importante:

- **Struttura e organizzazione:**
Offre un approccio strutturato e organizzato per lo sviluppo dei giochi. Include una raccolta di sistemi, classi e interfacce che lavorano insieme per creare la struttura di base di un gioco.
- **Logica e progressione di gioco:**
Include concetti predefiniti che aiutano a definire la logica, la progressione e l'organizzazione di un gioco.
- **Controllo del giocatore e dell'AI:**
Fornisce sistemi per gestire gli input del giocatore e le decisioni dei personaggi nel mondo di gioco. Include il controllo sia del giocatore che dell'IA, elementi essenziali per creare esperienze di gioco interattive e immersive.
- **Funzioni di utilità:**
Mette a disposizione una libreria di funzioni di utilità che assistono nelle operazioni comuni di gameplay e nelle interazioni. Queste funzioni semplificano la logica del gioco e migliorano l'efficienza nell'implementazione di diverse funzionalità.
- **Flessibilità e integrazione:**
È altamente flessibile e si integra in profondità con Unreal Engine. Utilizza pattern comuni di programmazione per i giochi e gestisce gran parte del lavoro di base, consentendo agli sviluppatori di concentrarsi sulla creazione dei loro giochi senza dover costruire un framework da zero.





Actors e Components





Actors e Components

In Unreal Engine, una classe **Actor** si riferisce a qualsiasi entità che può essere posizionata all'interno di un livello, che si tratti di una telecamera, una mesh statica o il personaggio del giocatore. Un Actor può subire trasformazioni come traslazione, rotazione e scalatura.

Gli Actor fungono da contenitori per classi specializzate note come **Componenti**, che svolgono diversi ruoli nel controllo del movimento, nel rendering e altro ancora.

Esistono tre tipi principali di componenti, ciascuno con uno scopo specifico all'interno di un Actor:

- Actor Components:
Contengono principalmente la logica del codice di un Actor. Gestiscono diverse funzionalità e interazioni senza avere una rappresentazione visiva.
- Scene Components:
Vengono utilizzati per posizionare e orientare altri componenti all'interno dell'Actor. Fungono da punti di riferimento per trasformazioni come traslazione, rotazione e scalatura, ma non hanno una presenza visibile e sono utilizzati principalmente a scopo organizzativo.
- Primitive Components:
Sono responsabili della rappresentazione visiva di un Actor all'interno di un livello. Possono essere renderizzati e interagire con i giocatori o con altri oggetti.

Combinando questi componenti, uno sviluppatore può creare Actor complessi e interattivi, che includono sia aspetti funzionali che visivi.



Game Instance

La classe *GameInstance* funge da gestore che opera in background (non è un actor di Unreal Engine). Una singola istanza viene creata all'avvio del motore e rimane attiva fino alla sua chiusura. Il suo scopo principale è tracciare i dati ed eseguire il codice necessario.

- Fornisce un hub centrale per gestire dati persistenti, come i sistemi di salvataggio.
- Funziona come manager per altri sottosistemi, offrendo un controllo comodo sul flusso del gioco.

Project - Maps & Modes

Default maps, game modes and other map-related settings.

These settings are saved in DefaultEngine.ini, which is currently writable.

Default Modes

Selected GameMode: **GameModeBase**

Default Pawn Class: **DefaultPawn**

HUD Class: **HUD**

Player Controller Class: **PlayerController**

Game State Class: **GameStateBase**

Player State Class: **PlayerState**

Spectator Class: **SpectatorPawn**

Advanced

Global Default Server Game Mode: **None**

Game Mode Map Profiles: 0 Active Elements

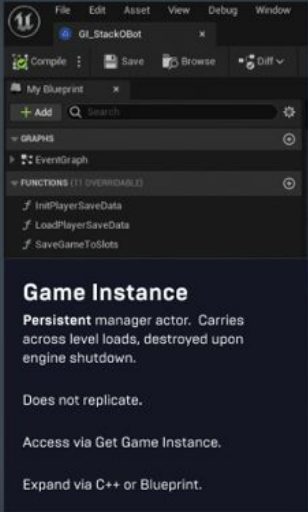
Game Mode Class Aliases: 0 Active Elements

Project Settings

Set **Gameplay Framework** defaults:

Game Mode (defines Player Pawn & Controller, HUD, Game & Player State and more), Game Instance and the starting Map.

Game Instance



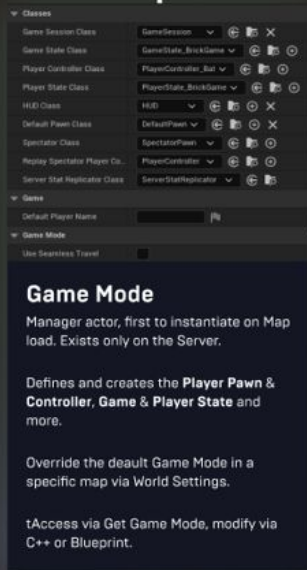
Game Instance

Persistent manager actor. Carries across level loads, destroyed upon engine shutdown.

Does not replicate.

Access via Get Game Instance.

Expand via C++ or Blueprint.



Game Mode

Manager actor, first to instantiate on Map load. Exists only on the Server.

Defines and creates the **Player Pawn & Controller, Game & Player State** and more.

Override the default Game Mode in a specific map via World Settings.

Access via Get Game Mode, modify via C++ or Blueprint.



Launch Engine



Load Level

In the context of the gameplay framework, all levels - traditional Maps or World Partition / Open World - load in the same manner.



Game Mode

Instantiates first, builds the Gameplay Framework



Game Mode e States

Diversamente dalla classe *GameInstance*, la classe *GameModeBase* (o la sua discendente diretta, *GameMode*) esiste solo in un singolo livello ed è creata subito dopo il caricamento del livello e la costruzione del mondo.

- Gestisce una sessione di gioco e ogni livello può avere una propria logica di game mode.
- Il suo ruolo principale è creare gli altri actor del framework.

Le classi *GameState* e *PlayerState* sono actor specializzati che svolgono un ruolo chiave nel tracciare lo stato del gioco e dei giocatori coinvolti.

- **GameState**: Memorizza e gestisce i dati rilevanti per tutti i giocatori di una partita.
 - **PlayerState**: Si concentra su uno specifico giocatore.
- Queste classi trovano applicazione principalmente nei giochi multiplayer, sia online che locali.

Game Mode & States



Controller



Load Level

In the context of the gameplay framework, all levels - traditional Maps or World Partition / Open World - load in the same manner.



Game Mode

Instantiates first, builds the Gameplay Framework



Player

A Player consists of a **Pawn** representing its physical presence and a **Controller** that determines their behavior.



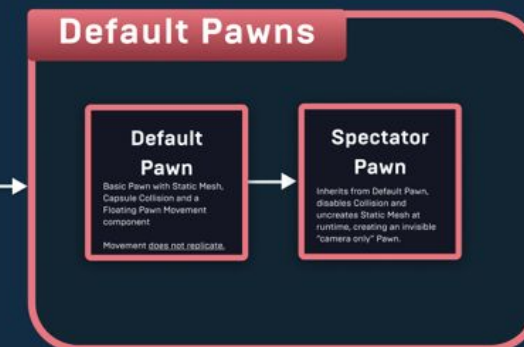
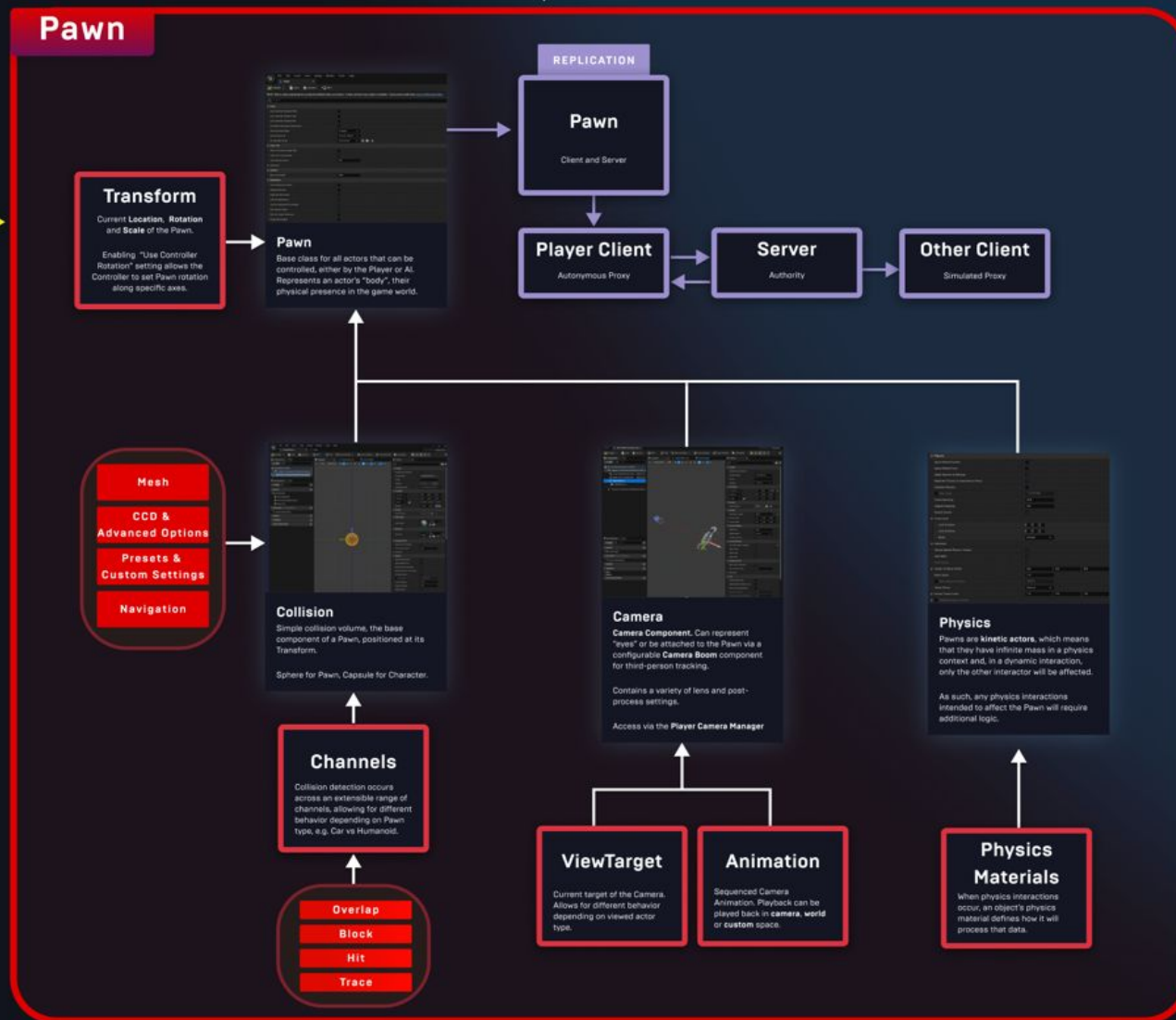
Pawn e Character

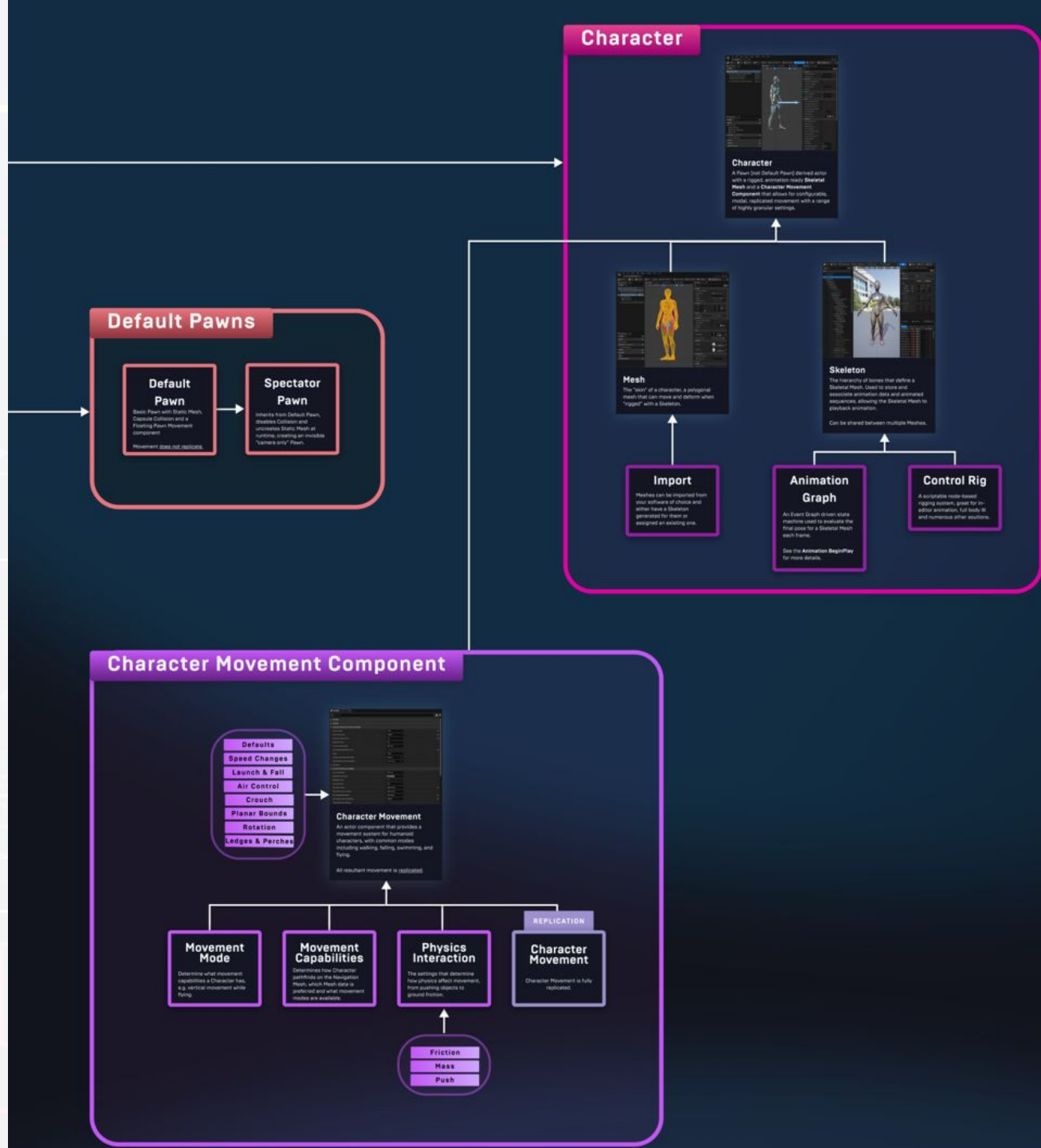
Il **Pawn** è la classe base per tutti gli actor che possono essere controllati da giocatori o entità AI nel mondo di gioco.

- Rappresenta fisicamente un'entità, gestendo interazioni come collisioni e fisica.
- Definisce spesso l'aspetto visivo dell'entità.

I **Character**, invece, estendono le funzionalità della classe Pawn.

- È progettata specificamente per rappresentare giocatori in un contesto verticale e consente azioni come camminare, correre, saltare e nuotare.
- Include funzionalità essenziali per la gestione del multiplayer.







Controller

La classe *Controller* governa la logica che determina le azioni di un giocatore nel mondo di gioco. Esistono due tipi principali di controller:

- **PlayerController**: Gestisce l'input di un giocatore umano, consentendo l'interazione con l'ambiente di gioco e migliorando l'esperienza complessiva.
- **AIController**: Governa le azioni di un'entità IA utilizzando strumenti come behavior tree, state tree, navigazione, ecc.

Entrambe le classi possono gestire un *Character* o un *Pawn* "possedendoli" durante l'esecuzione del gioco.

EOS

Services (EOS) in various systems as a range of services including voice chat, etc and many more.

PLICATION

State

Access Server and Client

PLICATION

er State

Access Server and Client

Input

Default in Unreal Engine 5.1



Enhanced Input
UE5 input management solution, with complex input handling, runtime control, remapping and comprehensive input data mining and processing features. Oversee the amount of device logic required for input handling. Create Input Actions, add to input mappings that can be edited as necessary via Blueprint or C++.

Player Input
UE4 input management solution. Project wide, with limited granularity and data processing. Define Action and Axis mappings in Project Settings, implement via Controller or Pawn logic.

RawInput
Player directly connect to interface with devices incompatible with Xinput, notably flight sticks and steering wheels. Allow Custom Input creation and mapping. Backwardly compatible with Player Input.

Behavior

Services
Recurring checks - query the Environment and Perception systems, update the Blackboard and more.

Decorators
Conditional, determining if nodes can execute. Attach to Composites to create branches.

Blackboard
Live data store utilized by the Behavior Tree.

Tasks
Behavior tree nodes that perform actions, either on the Blackboard or the controlled Pawn, e.g. Move or Shoot.

Composites
Root nodes, use it to build the branches of your Behavior Tree.

Behavior Tree
AI event-driven, task-based state machine that draws live data from a Blackboard and uses it to drive AI Behavior.

Navigation

Nav Agent
Simulated entity representing a smart character object, for calculating the NavMesh.

Smart Links
Allow AI to navigate between meshes in a custom manner, e.g. jumping.

Volume
Determines the boundaries of the generated Navigation Mesh.

Modifiers
Update the Navigation Mesh for weighted movement, object collision and more.

Nav Proxy Links
Logically connect NavMeshes with one another.

Navigation Mesh
A generated polygon mesh that defines navigable space for AI entities. Calculated by predefined agents within a preset bounding volume, with settings determining the level of detail.

Avoidance
Systems designed to prevent actors from colliding while navigating. Reciprocal Velocity Distances (RVD), Behavior Driven Manager. Both systems work independently and should be used judiciously in your project.

AI Movement
Provides pathfinding and navigation functionality for AI agents. Navigation Mesh and Avoidance systems can be utilized for improved behavior and additional fidelity.

Controller

Controller (Base Class)
Non-physical actors can possess a single instance of a Pawn or Pawn-derived class (such as Character) and control it - the "owner" is a Pawn's "owner". Controls receive event notifications for many Pawn events, allowing for the implementation of behavior logic and the overriding of default behavior. The Controller can be set to tick before a given Pawn, increasing input latency.

Player Controller
Controller interface, used to let a Player to control their possessed Pawn. Instantiates the HUD. Access via Get Player Controller. Modify via C++ or Blueprint.

AI Controller
Controller interface, used to control an AI possessed Pawn. Run only as Server. Access via Get Player Controller. Modify via C++ or Blueprint.

REPLICATION

Player Controller
Player Client to Server.

HUD

User Input
Input data is used to drive any interactive user interface elements.

HUD
Used to display information to Players and provide interactive user interface elements. Built using UMG (see the UMG BestPractices for more details) or another appropriate solution. Managed via Blueprint or C++ in a dedicated HUD actor, the Player Pawn, Controller or other combination of appropriate locations.

Data
Relevant data from the Gameplay Framework is fed to the HUD.

Viewport
Each client Player has their own non-physical viewport, or elements can be added to and removed from the viewport as necessary.

U

Player

A Player consists of a Pawn representing its physical presence and a Controller that determines their behavior.

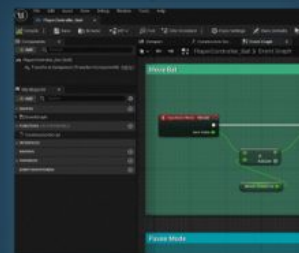
Controller

Controller (Base Class)

Non-physical actor, can possess a single instance of a Pawn or Pawn derived class (such as Character) and control it - the "brain" to a Pawn's "body."

Controllers receive event notifications for many Pawn events, allowing for the implementation of behavior logic and the overriding of default behavior.

The Controller can be set to tick before a given Pawn, minimizing input latency.



Player Controller

Controller subclass, used by a Player to control their possessed Pawn.

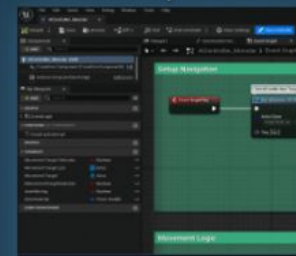
Instantiates the HUD.

Access via Get Player Controller.
Modify via C++ or Blueprint.

REPLICATION

Player Controller

Player Client to Server.



AI Controller

Controller subclass, used to control an AI possessed Pawn.

Runs only on Server.

Access via Get Player Controller.
Modify via C++ or Blueprint.

EOS

Services (EOS) in various systems as a range of services including voice chat, etc and many more.

PLICATION

State

Access Server and Client

PLICATION

er State

Access Server and Client

Input

Default in Unreal Engine 5.1



Enhanced Input
UE5 input management solution, with complex input handling, runtime control, remapping and comprehensive input data mining and processing features. Oversee the amount of device logic required for input handling. Create Input Actions, add to input mappings that can be edited as necessary via Blueprint or C++.

Player Input
UE4 input management solution. Project wide, with limited granularity and data processing. Define Action and Axis mappings in Project Settings, implement via Controller or Pawn logic.

RawInput
Player directly connect to interface with devices incompatible with Xinput, notably flight sticks and steering wheels. Allow Custom Input creation and mapping. Backwardly compatible with Player Input.

Behavior

Services
Recurring checks - query the Environment and Perception systems, update the Blackboard and more.

Decorators
Conditional, determining if nodes can execute. Attach to Composites to create branches.

Blackboard
Live data store utilized by the Behavior Tree.

Tasks
Behavior tree nodes that perform actions, either on the Blackboard or the controlled Pawn, e.g. Move or Shoot.

Composites
Root nodes, use it to build the branches of your Behavior Tree.

Behavior Tree
AI event-driven, task-based state machine that draws live data from a Blackboard and uses it to drive AI Behavior.

Navigation

Nav Agent
Simulated entity representing a smart character object, for calculating the NavMesh.

Smart Links
Allow AI to navigate between meshes in a custom manner, e.g. jumping.

Volume
Determines the boundaries of the generated Navigation Mesh.

Modifiers
Update the Navigation Mesh for weighted movement, object collision and more.

Nav Proxy Links
Logically connect NavMeshes with one another.

Navigation Mesh
A generated polygon mesh that defines navigable space for AI entities. Calculated by predefined agents within a preset bounding volume, with settings determining the level of detail.

Avoidance
Systems designed to prevent actors from colliding while navigating. Reciprocal Velocity Distances (RVD), Behavior Driven Manager. Both systems work independently and should be used judiciously in your project.

AI Movement
Provides pathfinding and navigation functionality for AI agents. Navigation Mesh and Avoidance systems can be utilized for improved behavior and additional fidelity.

Controller

Controller (Base Class)
Non-physical actors can possess a single instance of a Pawn or Pawn-derived class (such as Character) and control it - the "owner" is a Pawn's "owner". Coordinates runtime event notifications for many Pawn events, allowing for the implementation of behavior logic and the overriding of default behavior. The Controller can be set to tick before a given Pawn, increasing input latency.

Player Controller
Controller activities, used to a Player to control their possessed Pawn. Instantiates the HUD. Access via Get Player Controller. Modify via C++ or Blueprint.

AI Controller
Controller activities, used to control an AI possessed Pawn. Run only as Service. Access via Get Player Controller. Modify via C++ or Blueprint.

REPLICATION

Player Controller
Player Client to Server.

HUD

User Input
Input data is used to drive any interactive user interface elements.

HUD
Used to display information to Players and provide interactive user interface elements. Built using UMG (see the UMG BestPractices for more details) or another appropriate solution. Managed via Blueprint or C++ in a dedicated HUD actor, the Player Pawn, Controller or other combination of appropriate locations.

Data
Relevant data from the Gameplay Framework is fed to the HUD.

Viewport
Each client Player has their own non-physical viewport, or elements can be added to and removed from the viewport as necessary.

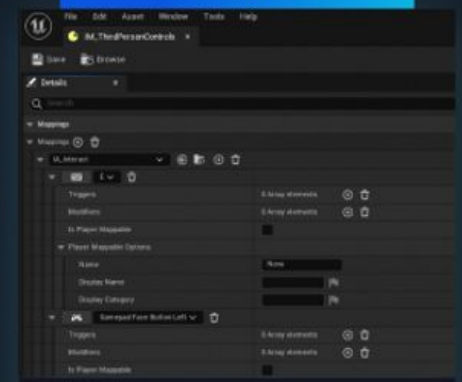
U

Player

A Player consists of a Pawn representing its physical presence and a Controller that determines their behavior.

Input

Default In
Unreal Engine 5.1

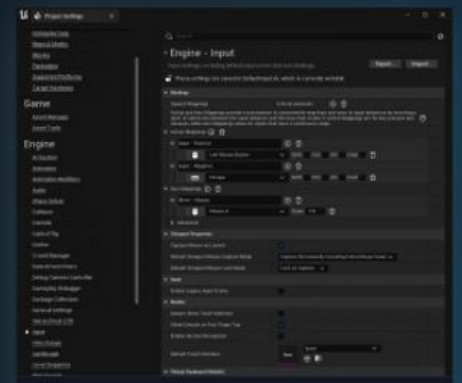


Enhanced Input

UE5 input management solution, with complex input handling, runtime control remapping and comprehensive input data filtering and processing features. Drastically reduces the amount of bespoke logic required for input handling.

Create Input Actions, add to Input Mappings that can be applied as necessary via Blueprint or C++.

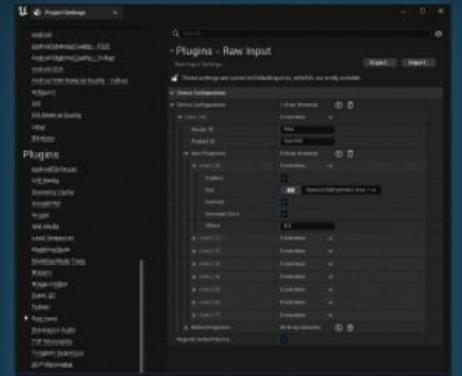
Backwards compatible with **Player Input**.



Player Input

UE4 input management solution. Project wide, with limited granularity and data processing.

Define Action and Axis mappings in Project Settings, implement via Controller or Pawn logic.



RawInput

Plugin allowing Unreal to interface with devices incompatible with Xinput, notably flight sticks and steering wheels.

Allows Custom Input creation and mapping.

Behavior

Services

Recurring checks - query the Environment and Perception systems, update the Blackboard and more.

Decorators

Conditionals, determining if nodes can execute. Attach to Composites to create branches.

Blackboard

Live data store utilized by the Behavior Tree.

Tasks

Behavior tree nodes that perform actions, either on the Blackboard or the controlled Pawn, e.g. Move or Shoot.

Composites

Root node, use it to build the branches of your Behavior Tree.



Input

Default to Unreal Engine 5.1

Enhanced Input

Create input Actions, and to input Perception that can be extended and modified via Blueprint or C++.

Supports compatibility with Player Input

Controller

Controller (Base Class)

Non-abstract Actor that represents a single instance of a Pawn or Pawn component. It is the base class for all Pawn components.

Controllers are used to manage the state of a Pawn and its components. They are responsible for handling input, movement, and other game logic.

The Controller can be set to use a specific Pawn Input, controlling their actions.

Smart Links

Allow AI to navigate between nodes in a Behavior Tree.

Nav Proxy Links

Connect to a Nav Proxy to get the next location to move to.

Avoidance

Prevents the AI from moving into areas that are blocked or dangerous.

Regional Velocity Response (RVR)

Adjusts the AI's velocity based on the region it is in.

AI Movement

Provides a simple and easy way to move the AI.

Navigation Mesh and Pathfinding can be used to find the best route to a destination.

HUD

Used to display information to the Player.

Can be used to show the AI's current state or other game information.

Viewport

Used to display the game world to the Player.

Can be used to show the AI's current state or other game information.

Data

Used to store game data.

Can be used to store the AI's current state or other game information.



Player

Navigation

Input

Default to Unreal Engine 5.1

Navigation is a system that allows AI agents to move through a level. It is a complex system that involves many different components, including the Nav Agent, Smart Links, Volume, Modifiers, Nav Proxy Links, Navigation Mesh, Avoidance, and AI Movement.

Enhanced Input

Use the enhanced input system to provide more detailed information to the Nav Agent. This system allows you to define custom input for specific AI agents, such as their speed, acceleration, and other movement parameters.

Create input actions and assign them to specific AI agents. You can also use the enhanced input system to define custom input for specific AI agents, such as their speed, acceleration, and other movement parameters.

See the documentation for more information on the enhanced input system.

Controller

Controller (Base Class)

The Controller is the base class for all AI agents. It defines the basic behavior and movement of the AI agents. The Controller is responsible for managing the AI agent's state, including its position, velocity, and other movement parameters.

The Controller is responsible for managing the AI agent's state, including its position, velocity, and other movement parameters. It also handles the AI agent's input and output, and manages the AI agent's behavior.

See the documentation for more information on the Controller class.

Nav Agent

Simulated entity representing a preset character shape, for calculating the NavMesh.

Smart Links

Allow AI to navigate between meshes in a custom manner, e.g. jumping

Volume

Determines the boundaries of the generated Navigation Mesh.

Modifiers

Update the Navigation Mesh for weighted movement, object collision and more.

Nav Proxy Links

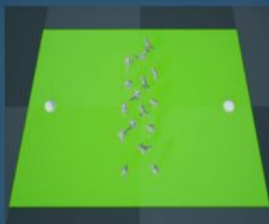
Logically connect NavMeshes with one another.



Navigation Mesh

A generated polygon mesh that defines navigable space for AI entities. Calculated by predefined agents within a placed bounding volume, with settings determining the level of dynamism.

Multiple agents allow for multiple navmeshes to accommodate varying actor scale.



Avoidance

Systems designed to prevent actors from colliding while navigating.

Unreal comes with two potential options - **Reciprocal Velocity Obstacles (RVO)** and **Detour Crowd Manager**.

Both systems work independently and should be used exclusively in your project.

AI Movement

Provides pathfinding and navigation functionality for AI agents.

Navigation Mesh and Avoidance systems can be utilized for improved behavior and additional fidelity.

Smart Links

Allow AI to navigate between meshes in a custom manner, e.g. jumping

Nav Proxy Links

Logically connect NavMeshes with one another.

Avoidance

Systems designed to prevent actors from colliding while navigating.

Unreal comes with two potential options - **Reciprocal Velocity Obstacles (RVO)** and **Detour Crowd Manager**.

Both systems work independently and should be used exclusively in your project.

AI Movement

Provides pathfinding and navigation functionality for AI agents.

Navigation Mesh and Avoidance systems can be utilized for improved behavior and additional fidelity.

Data

Provides information to the AI agents. This system allows you to define custom input for specific AI agents, such as their speed, acceleration, and other movement parameters.

Viewport

Each actor's position and other movement parameters. The viewport is the area where the AI agents are visible to the player.



Player

A Player is a character that can move through the level and interact with the environment. The Player is the main character in the game and is controlled by the player.

EOS

Services (EOS) in various systems as a range of services including voice chat, etc and many more.

PLICATION

State

Access Server and Client

PLICATION

er State

Access Server and Client

Input

Default in Unreal Engine 5.1



Enhanced Input
UE5 input management solution, with complex input handling, runtime control, remapping and comprehensive input data mining and processing features. Dynamically replaces the amount of bespoke logic required for input handling. Create Input Actions, add to input mappings that can be edited as necessary via Blueprint or C++.

Player Input
UE4 input management solution. Project wide, with limited granularity and data processing. Define Action and Axis mappings in Project Settings, implement via Controller or Pawn logic.

RawInput
Player allowing direct to interface with devices incompatible with Xinput, notably flight sticks and steering wheels. Allows Custom Input creation and mapping. Backwardly compatible with Player Input.

Behavior

Services
Recurring checks - query the Environment and Perception systems, update the Blackboard and more.

Decorators
Conditional, determining if nodes can execute. Attach to Composites to create branches.

Blackboard
Live data store utilized by the Behavior Tree.

Tasks
Behavior tree nodes that perform actions, either on the Blackboard or the controlled Pawn, e.g. Move or Shoot.

Composites
Root nodes, use it to build the branches of your Behavior Tree.

Behavior Tree
An event-driven, task-based state machine that draws live data from a Blackboard and uses it to drive AI behavior.

Navigation

Nav Agent
Simulated entity representing a smart character object, for calculating the NavMesh.

Smart Links
Allow AI to navigate between meshes in a custom manner, e.g. jumping.

Volume
Determines the boundaries of the generated Navigation Mesh.

Modifiers
Update the Navigation Mesh for weighted movement, object collision and more.

Nav Proxy Links
Logically connect NavMeshes with one another.

Navigation Mesh
A generated polygon mesh that defines navigable space for AI entities. Calculated by predefined agents within a preset bounding volume, with settings determining the level of detail.

Avoidance
Systems designed to prevent actors from colliding while navigating. Reciprocal Velocity Distances (RVD), Behavior Driven Manager. Both systems work independently and should be used judiciously in your project.

AI Movement
Provides pathfinding and navigation functionality for AI agents. Navigation Mesh and Avoidance systems can be utilized for improved behavior and additional fidelity.

Controller

Controller (Base Class)
Non-physical actors can possess a single instance of a Pawn or Pawn-derived class (such as Character) and control it - the "owner" is a Pawn's "owner". Controls receive event notifications for many Pawn events, allowing for the implementation of behavior logic and the overriding of default behavior. The Controller can be set to tick before a given Pawn, increasing input latency.

Player Controller
Controller activities, used to a Player to control their possessed Pawn. Instantiates the HUD. Access via Get Player Controller. Modify via C++ or Blueprint.

AI Controller
Controller activities, used to control an AI possessed Pawn. Run only as Server. Access via Get Player Controller. Modify via C++ or Blueprint.

REPLICATION

Player Controller
Player Client to Server.

HUD

User Input
Input data is used to drive any interactive user interface elements.

HUD
Used to display information to Players and provide interactive user interface elements. Built using UMG (see the UMG BestPractices for more details) or another appropriate solution. Managed via Blueprint or C++ in a dedicated HUD actor, the Player Pawn, Controller or other combination of appropriate locations.

Data
Relevant data from the Gameplay Framework is fed to the HUD.

Viewport
Each client Player has their own non-physical viewport, or elements can be added to and removed from the viewport as necessary.

U

Player

A Player consists of a Pawn representing its physical presence and a Controller that determines their behavior.

HUD

User Input

Input data is used to drive any interactive User Interface elements.

HUD

Used to display information to Players and provide interactive User Interface elements.

Built using **UMG** (see the **UMG BeginPlay** for more details) or another appropriate solution.

Managed via Blueprint or C++ in a standalone HUD actor, the Player Pawn, Controller or other combination of appropriate locations.

Viewport

Each client Player has their own non-replicated viewport. UI elements can be added to and removed from this viewport as necessary

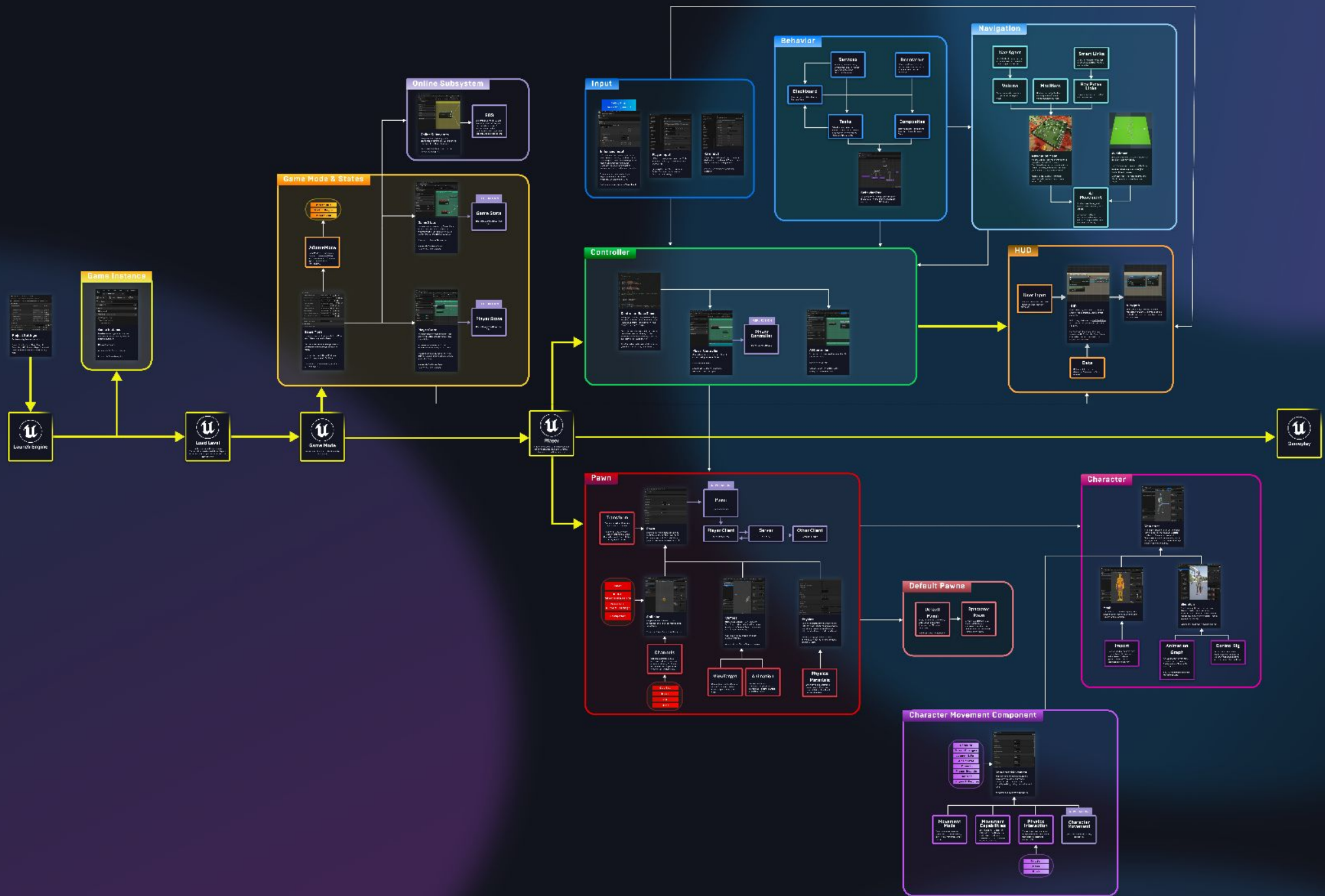
Data

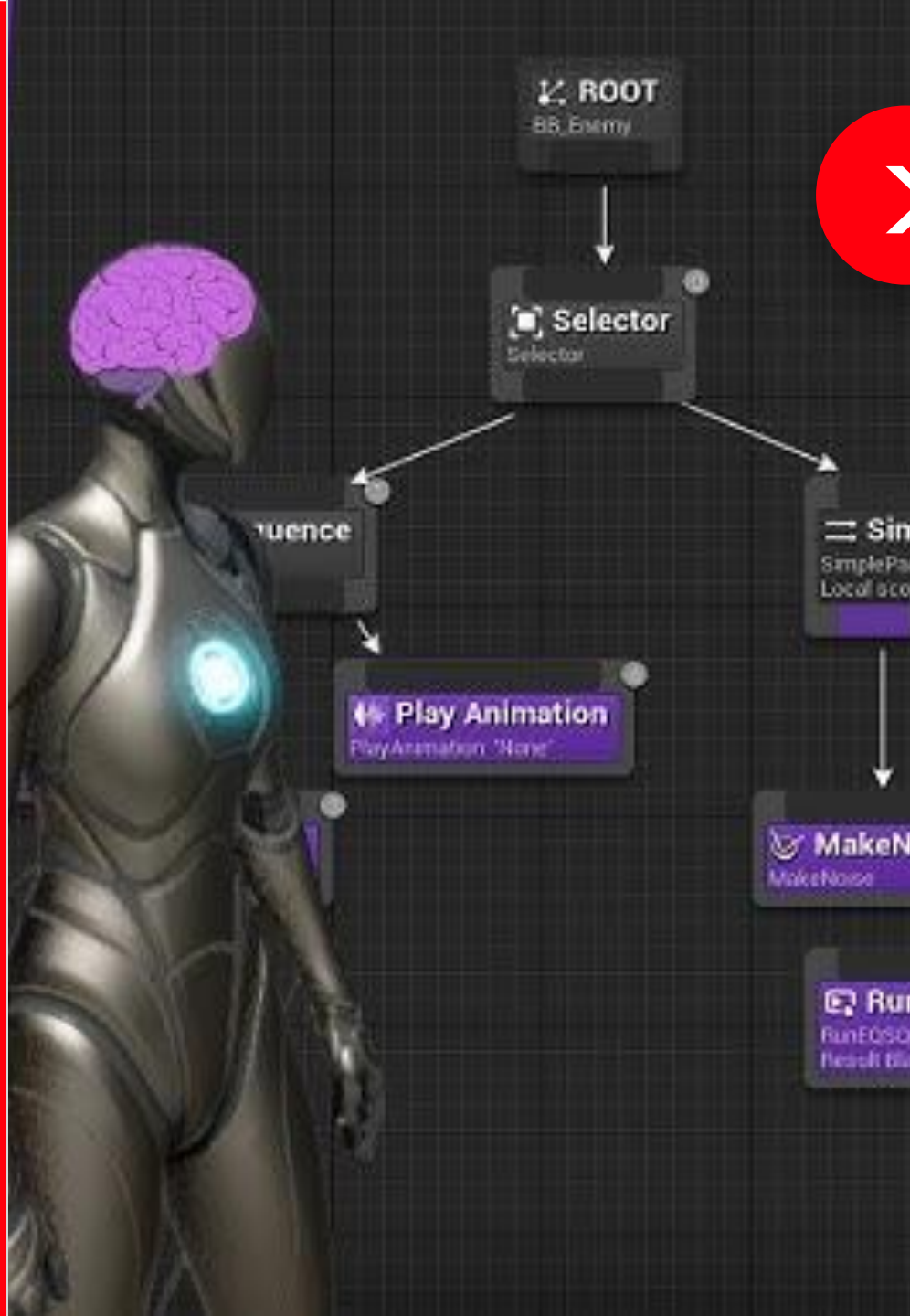
Relevant data from the Gameplay Framework is fed to the HUD.



Player

A Player represents all a Player's resources, including the physical component and all its abilities. Each Player has a set of abilities.





Unreal Engine AI System

Considerando la potenza del framework descritto in precedenza, non sorprende che Unreal Engine offra un sistema di intelligenza artificiale completo e robusto.

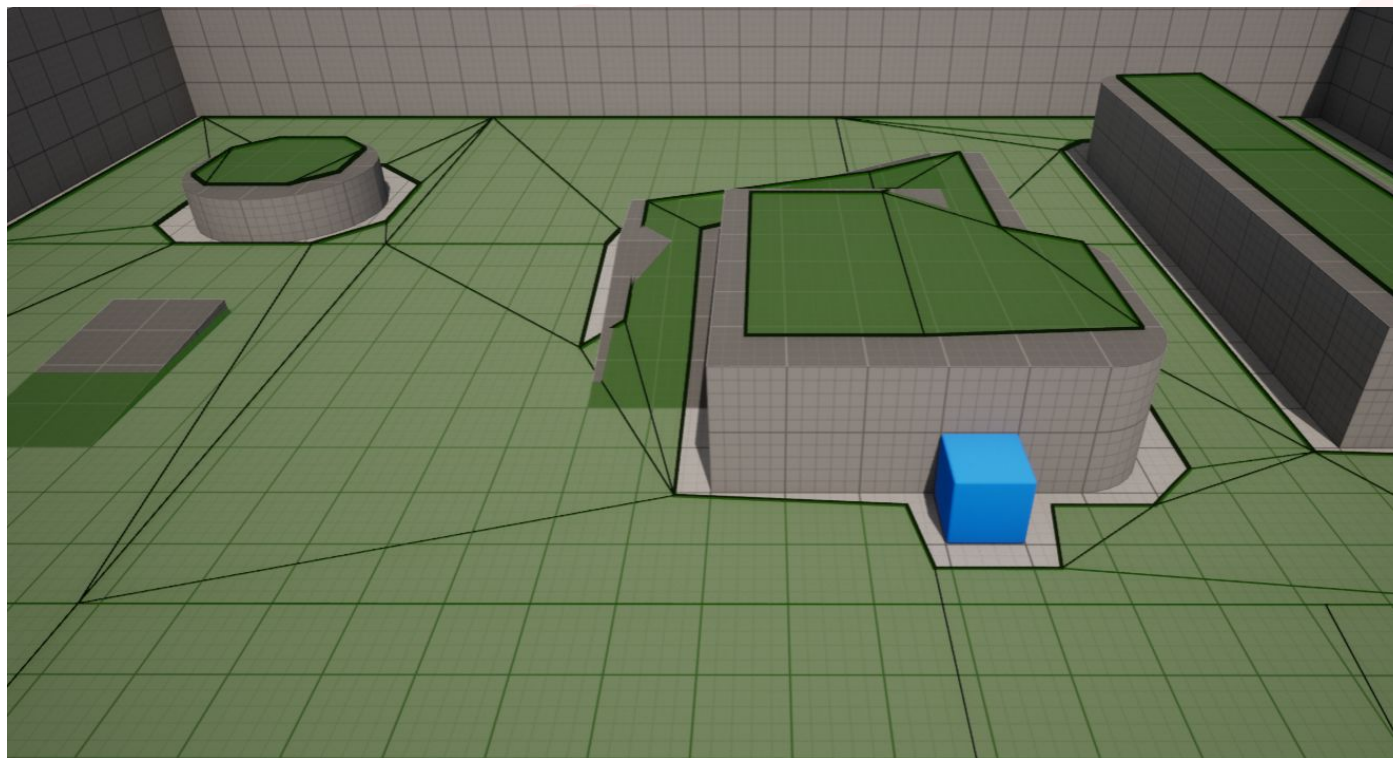
In questa sezione, presenteremo l'ampia gamma di strumenti disponibili per i programmatori di AI in Unreal Engine, accompagnati da una breve descrizione delle loro principali caratteristiche



Navigation System

Il **Navigation System** di Unreal Engine consente alle entità AI, chiamate agenti, di muoversi all'interno di un livello utilizzando algoritmi di pathfinding.

Il sistema genera una **Nav Mesh** basata sulla geometria presente nel livello tramite le collisioni.



Questa mesh viene successivamente suddivisa in tile, che a loro volta sono partizionati in poligoni, formando così un grafo. Gli agenti nel sistema utilizzano questo grafo per navigare verso le loro destinazioni.

I poligoni hanno un costo assegnato, che aiuta gli agenti a determinare il percorso ottimale in base al costo complessivo più basso.



Navigation System

Inoltre, il Navigation System include una serie di componenti e impostazioni regolabili per modificare il processo di generazione della nav mesh.

Queste modifiche possono includere variazioni dei costi dei poligoni, influenzando il comportamento di navigazione degli agenti all'interno del livello.

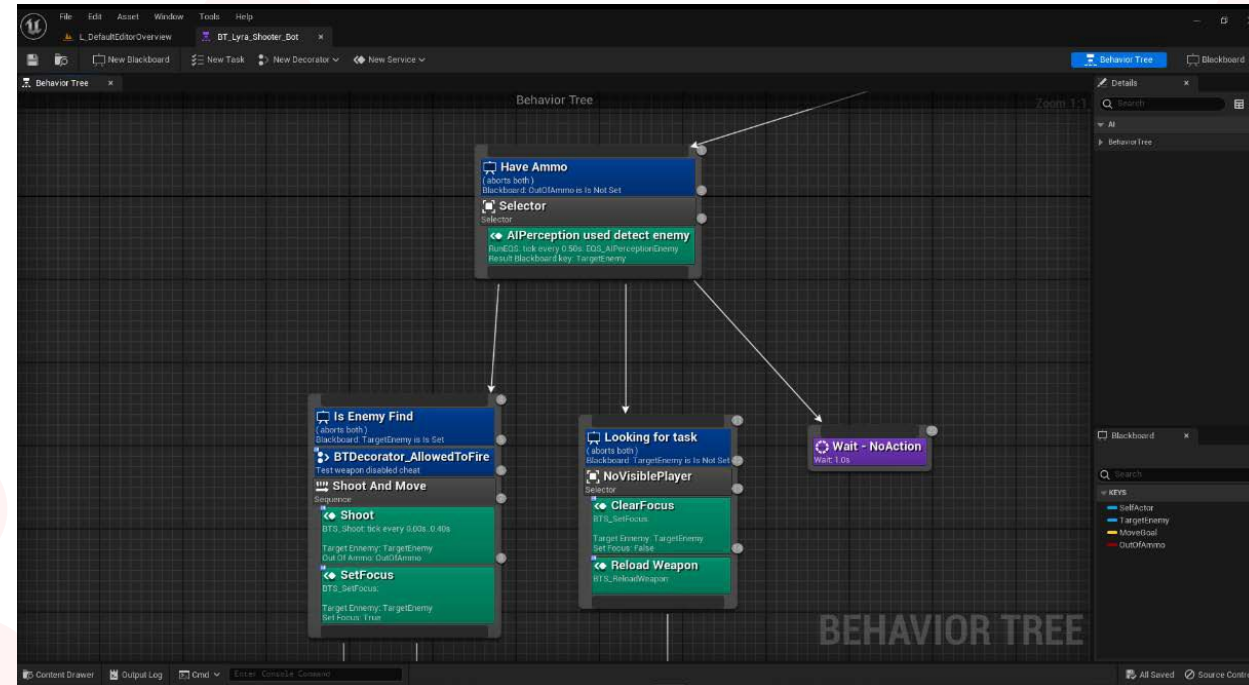
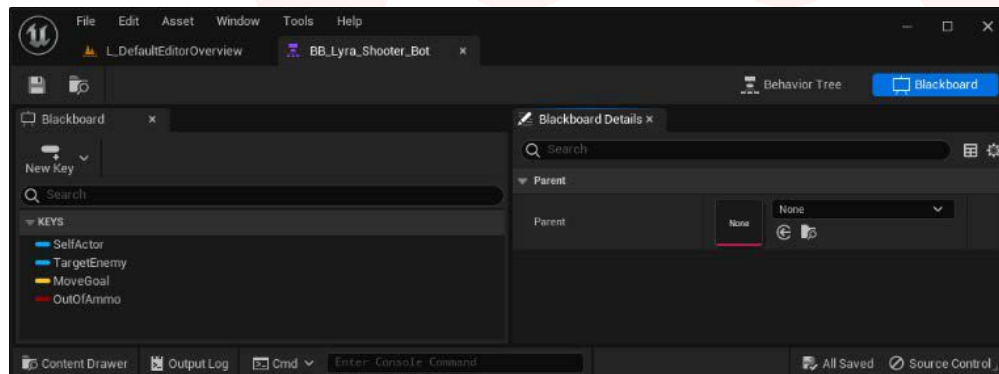
Infine, il sistema permette di connettere aree non contigue all'interno della nav mesh, come piattaforme e ponti, facilitando una navigazione fluida attraverso questi elementi spaziali.





Behaviour Trees

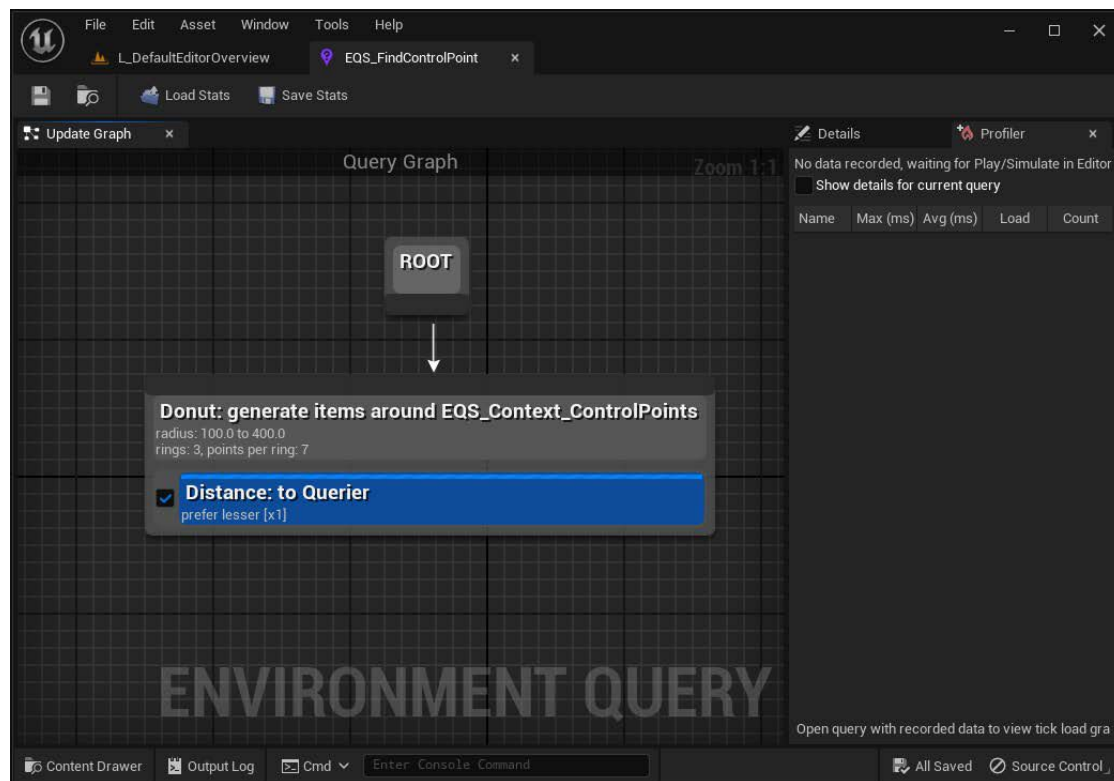
In Unreal Engine, i **Behaviour Trees** sono uno strumento prezioso per creare intelligenze artificiali per gli NPC nei tuoi giochi. La funzione principale di un *asset Behavior Tree* è eseguire ramificazioni contenenti istruzioni logiche. In Unreal Engine, i Behaviour Tree vengono creati in modo molto simile ai Blueprints, utilizzando una sorta di scripting visuale. Questo consiste nell'aggiungere e connettere una sequenza di nodi con funzionalità specifiche per formare un grafo del behaviour tree.



Per determinare quali ramificazioni devono essere eseguite, il behaviour tree si affida a un altro asset noto come **Blackboard**, che funge da "cervello" del behaviour tree stesso.



Behaviour Trees - EQS



Il **Environment Query System** (EQS)

raccoglie dati dall'ambiente, consentendo alle AI di interrogare tali dati utilizzando vari test. Questo processo permette di selezionare l'elemento che meglio corrisponde alla domanda posta.

Le query possono essere richiamate da un Behaviour Tree e utilizzate per prendere decisioni su come procedere in base ai risultati dei test eseguiti.



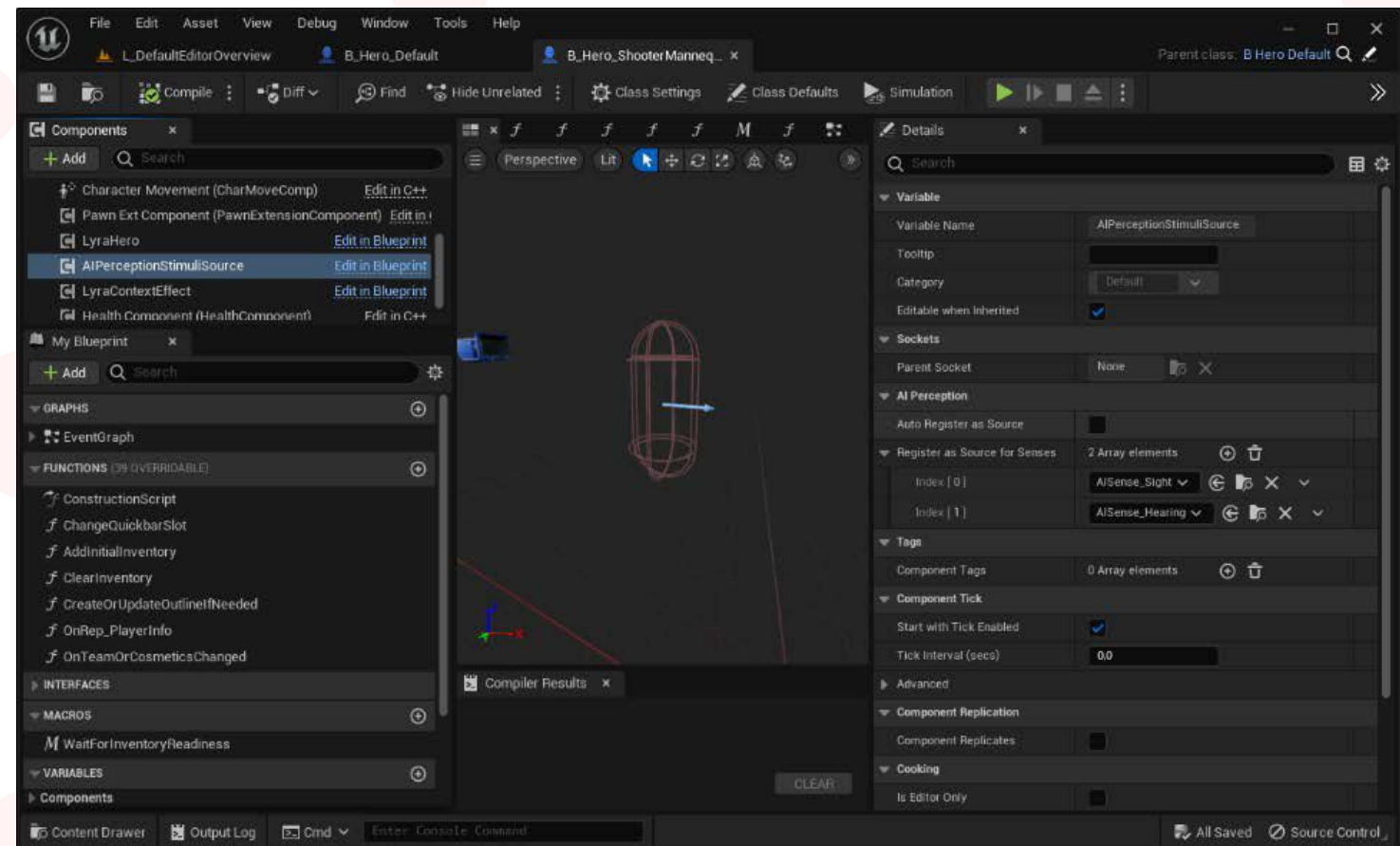
AI Perception System

L'**AI Perception System** offre un altro metodo per permettere ai Pawn di ricevere dati dall'ambiente, come la provenienza dei rumori o l'individuazione di oggetti visibili.

Questo sistema consente di generare consapevolezza per l'AI fornendole dati sensoriali.

Il sistema permette alle sorgenti di dati di creare stimoli, così che i *listener* possano essere aggiornati periodicamente su di essi.

Viene utilizzato per abilitare le capacità sensoriali dell'AI nei giochi e può reagire a una vasta gamma di sensori personalizzabili.





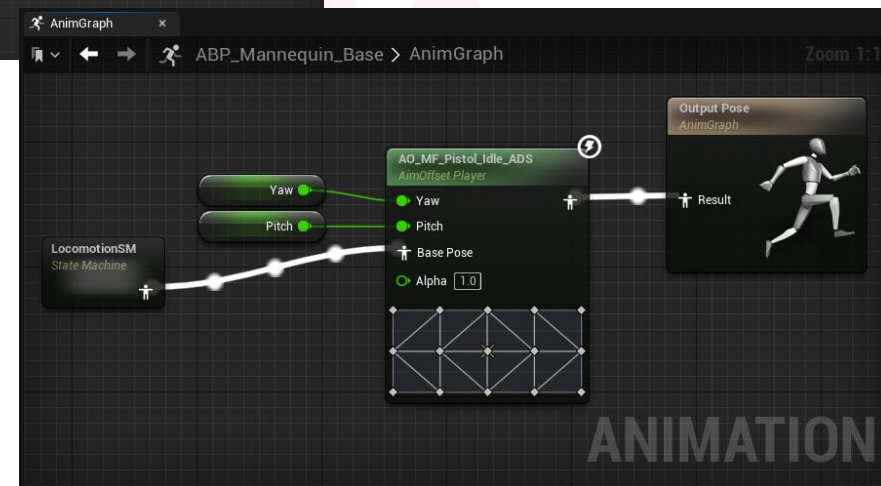
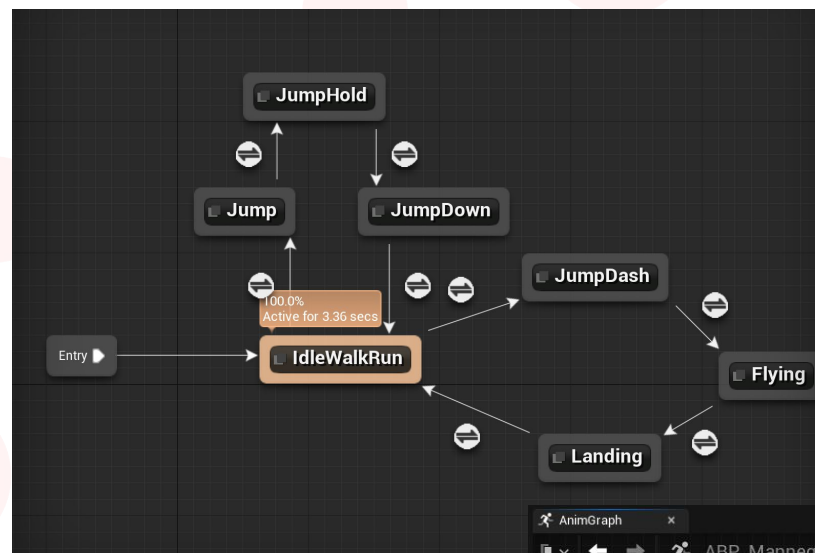
AI Animation Blueprint

L'**Animation Blueprint** è uno strumento avanzato per gestire le animazioni degli agenti controllati dall'AI in Unreal Engine.

Combina il comportamento dinamico dell'AI con transizioni fluide tra le animazioni, migliorando l'esperienza di gioco.

ABP in pillole:

- Gestione dinamica (real time) delle animazioni
- Integrazione con Behavior Trees
- Blending delle animazioni tramite l'accesso a variabili
- Supporto per IK per un movimento realistico delle parti del corpo.
- Ottimizzazione e prestazioni per ottenere animazioni performanti per un'IA reattiva e fluida.

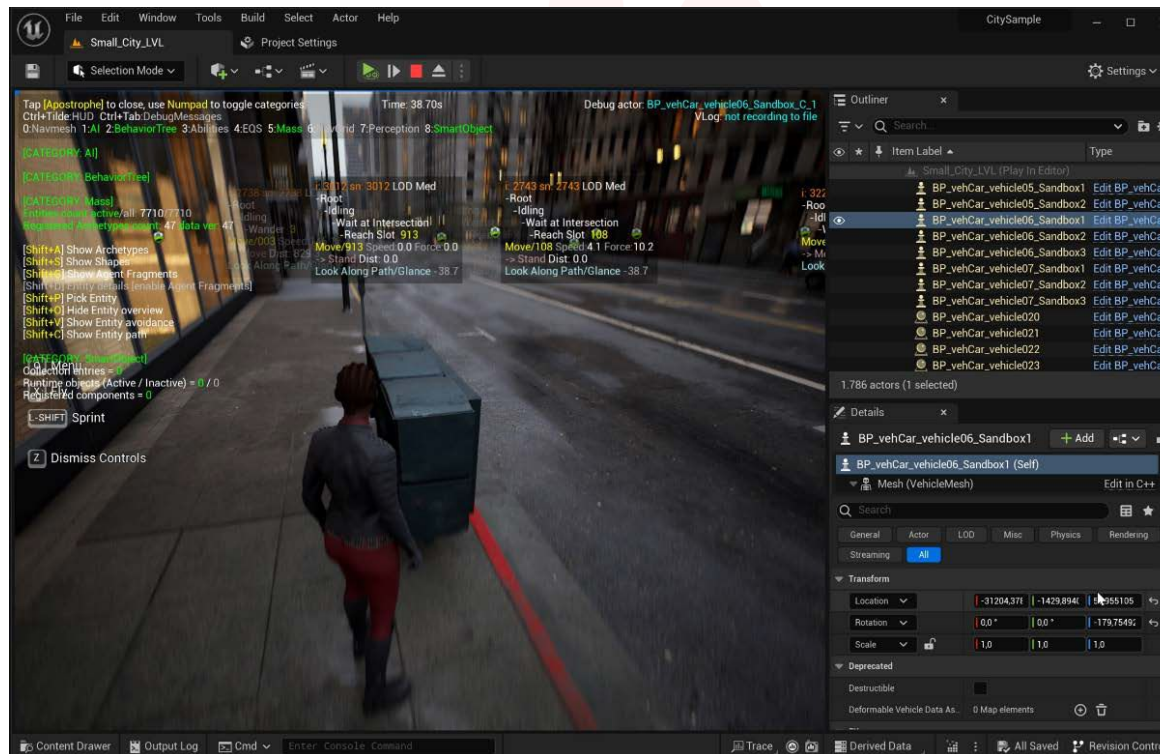




AI Debugging

Nessun framework serio sarebbe completo senza un sistema di **debug**. Il debugging è un aspetto essenziale dello sviluppo software, in quanto consente agli sviluppatori di identificare e correggere errori o bug nel loro codice.

Gioca un ruolo cruciale nel garantire l'affidabilità e la funzionalità del framework.



Per questo motivo, Unreal Engine offre un arsenale completo di strumenti e funzionalità per assistere gli sviluppatori nel debug delle AI, inclusi strumenti di debug visivo, visualizzazione dei behavior tree e modalità di simulazione dell'AI.

Questi strumenti permettono agli sviluppatori di ispezionare e modificare in tempo reale il comportamento delle AI, identificare problemi come errori di pathfinding o decisioni erratiche, e apportare le modifiche necessarie per migliorare le prestazioni complessive dell'AI all'interno dell'ambiente di gioco.



Alcune di queste funzionalità sono ancora in fase **sperimentale** o di rilascio **beta**, quindi devono essere utilizzate con cautela.

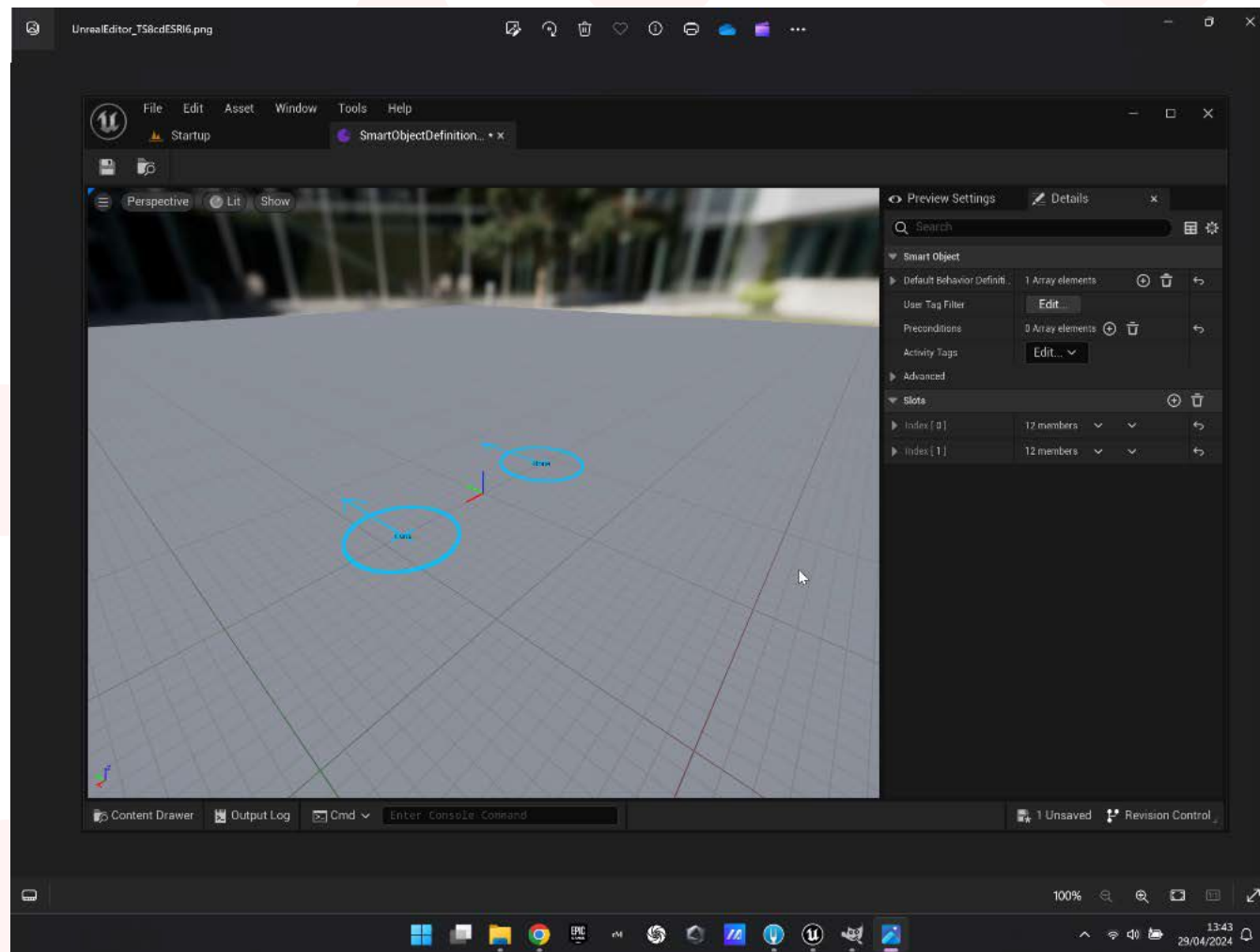


AI - Advanced Topics

Smart Objects: rappresentano un insieme di attività nel livello che possono essere utilizzate tramite un sistema di prenotazione che garantisce che solo un agente AI possa utilizzare uno smart object alla volta, impedendo ad altri agenti di utilizzarlo fino a quando non diventa di nuovo disponibile.

Questi oggetti sono posizionati su un livello e ci possono interagire sia gli agenti AI che i giocatori.

Gli smart objects contengono tutte le informazioni necessarie per queste interazioni e possono essere interrogati durante l'esecuzione utilizzando filtri dedicati.

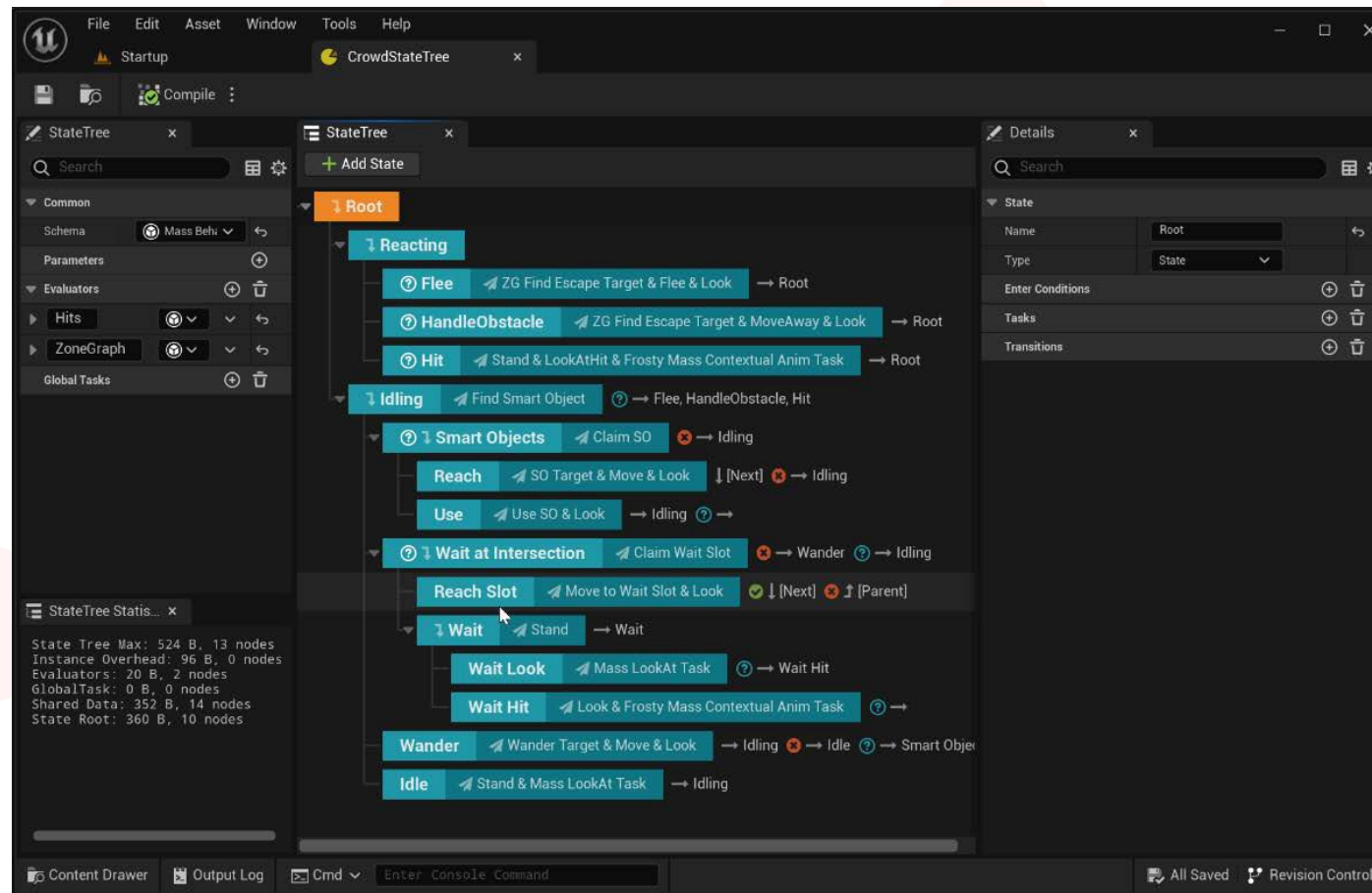




AI - Advanced Topics

State Tree: è una macchina a stati gerarchica versatile che integra alcune caratteristiche dei Behaviour Trees con altre provenienti dalle macchine a stati (FSM).

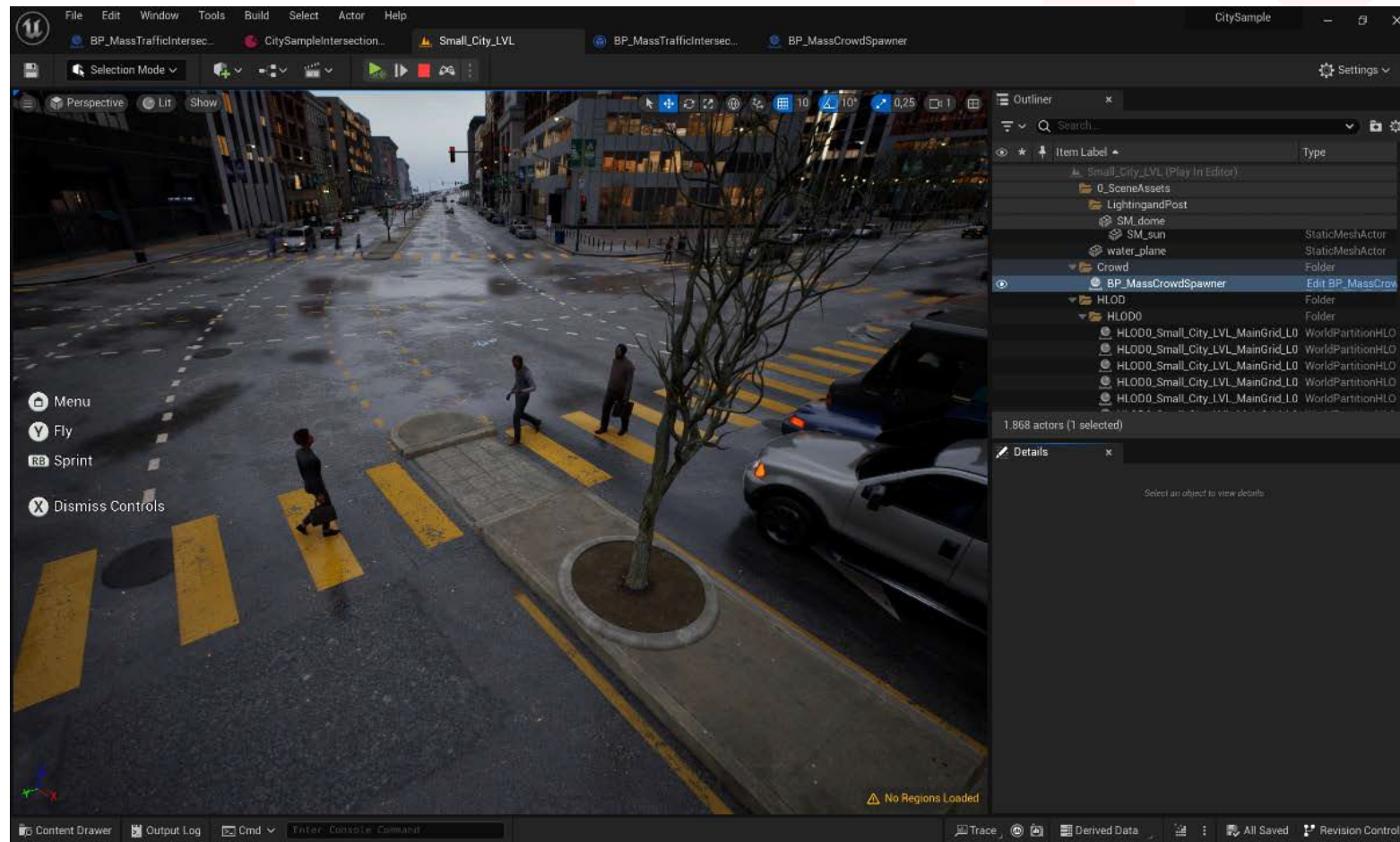
Con questo sistema, organizzato in una struttura ad albero, gli sviluppatori saranno in grado di creare logiche altamente performanti che rimangono strutturate e adattabili.





AI - Advanced Topics

Mass Entity: è un framework focalizzato sul gameplay per calcoli data-oriented, ovvero che si concentra sull'organizzazione e sull'elaborazione dei dati anziché sui comportamenti o le entità stesse, fornendo un paradigma per la gestione degli elementi con behaviour nel gioco.



È progettato per gestire un grande numero di entità e facilitare i controlli del comportamento sia per i modelli scheletrici che per quelli statici.

